

SKILL WEEK-5

Session-9

Task1: To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

```
madhulika@LAPTOP-D2M8NKBH:~$ mkdir -p ~/OSSP/skill/week5/task1
madhulika@LAPTOP-D2M8NKBH:~$ cd ~/OSSP/skill/week5/task1
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ nano task1.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ gcc task1.c -o T1
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ ./T1
=== Skill Week 5 - Task 1 ===
Directory Navigation

Current Working Directory: /home/madhulika/OSSP/skill/week5/task1

Enter directory to change to: /tmp
Path '/tmp' is valid.
Directory changed successfully.
Updated Working Directory: /tmp

Previous Directory: /home/madhulika/OSSP/skill/week5/task1
Returning to previous directory...
Final Working Directory: /home/madhulika/OSSP/skill/week5/task1

Directory navigation completed successfully.
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ ./T1
=== Skill Week 5 - Task 1 ===
Directory Navigation

Current Working Directory: /home/madhulika/OSSP/skill/week5/task1

Enter directory to change to: /abc123
Error: Path '/abc123' does not exist.
Path validation: No such file or directory
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$
```

```
#include <stdio.h>
#include <unistd.h>
#include <limits.h>
#include <errno.h>
#include <string.h>

int main() {
    char current_dir[PATH_MAX];
    char previous_dir[PATH_MAX];
    char target_dir[PATH_MAX];

    printf("=== Skill Week 5 - Task 1 ===\n");
    printf("Directory Navigation\n\n");

    /* Get current working directory */
    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("Error getting current directory");
        return 1;
    }

    printf("Current Working Directory: %s\n", current_dir);

    /* Store previous directory */
    strcpy(previous_dir, current_dir);

    /* Ask for target directory */
    printf("\nEnter directory to change to: ");
    scanf("%s", target_dir);

    /* Validate path */
    if (access(target_dir, F_OK) != 0) {
        printf("Error: Path '%s' does not exist.\n", target_dir);
        perror("Path validation");
        return 1;
    }

    printf("Path '%s' is valid.\n", target_dir);
```

```

}

printf("Directory changed successfully.\n");

/* Update working directory */
if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
    perror("Error getting updated directory");
    return 1;
}

printf("Updated Working Directory: %s\n", current_dir);

/* Display previous directory */
printf("\nPrevious Directory: %s\n", previous_dir);

/* Return to previous directory */
printf("Returning to previous directory...\n");

if (chdir(previous_dir) != 0) {
    printf("Error: Could not return to previous directory.\n");
    perror("chdir");
    return 1;
}

/* Verify navigation */
if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
    perror("Error getting final directory");
    return 1;
}

printf("Final Working Directory: %s\n", current_dir);

printf("\nDirectory navigation completed successfully.\n");

return 0;
}

```

Observation: The program successfully performs directory navigation using Linux system calls. It validates the given path before changing the directory, changes the current working directory using `chdir()`, and obtains the updated directory using `getcwd()`. The previous working directory is stored so that the program can return to it. Error handling is also implemented using `perror()` when an invalid or inaccessible path is provided. The program successfully navigates to a valid directory and returns to the previous directory.

Task2: To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```

madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5$ mkdir -p task2
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5$ cd task2
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task2$ nano task2.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task2$ gcc task2.c -o T2
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task2$ ./T2
=== Skill Week 5 - Task 2 ===
Built-in Command Dispatch

myshell> pwd
Current Directory: /home/madhulika/OSSP/skill/week5/task2
myshell> ls
T2 task2.c
myshell> cd /tmp
Directory changed successfully.
myshell> pwd
Current Directory: /tmp
myshell> echo Hello Week 5
Hello Week 5
myshell> hello
Error: Invalid command 'hello'
myshell> exit
Exiting shell...
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task2$

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 100

/* Built-in command functions */
void execute_pwd() {
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("Current Directory: %s\n", cwd);
    else
        perror("getcwd");
}

void execute_cd(char *path) {
    if (path == NULL) {
        printf("cd: missing directory\n");
        return;
    }

    if (chdir(path) != 0)
        perror("cd");
    else
        printf("Directory changed successfully.\n");
}

void execute_ls() {
    pid_t pid = fork();

    if (pid == 0) {
        execlp("ls", "ls", NULL);
        perror("ls");
        exit(1);
    }
}

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```

        printf("Exiting Shell...\n");
        break;
    }

    /* Handle cd */
    if (strcmp(command, "cd") == 0) {
        execute_cd(argument);
        continue;
    }

    /* Handle echo */
    if (strcmp(command, "echo") == 0) {
        execute_echo(argument);
        continue;
    }

    /* Search dispatch table */
    int found = 0;

    for (int i = 0; i < command_count; i++) {
        if (strcmp(command, dispatch_table[i].name) == 0) {
            found = 1;

            if (dispatch_table[i].function != NULL)
                dispatch_table[i].function();

            break;
        }
    }

    if (!found)
        printf("Error: Invalid command '%s'\n", command);
}

return 0;
}

```

^{^G} Help ^{^O} Write Out ^{^F} Where Is ^{^K} Cut ^{^T} Execute
^{^X} Exit ^{^R} Read File ^{^_} Replace ^{^U} Paste ^{^J} Justify

Observation: The program successfully implements a command dispatch mechanism for handling built-in commands. It identifies commands such as `pwd`, `ls`, `cd`, and `echo` and executes the corresponding operations. The dispatch table is used to associate commands with their functions. The `cd` command changes the working directory within the same process, while `ls` is executed using `fork()` and `execvp()`. Invalid commands are detected and an error message is displayed. The `exit` command terminates the shell successfully.

Session-10

Task1: To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

```
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5$ mkdir -p task1
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5$ cd task1
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ nano task1.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ nano directory_state.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ gcc directory_state.c -o
T1
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ ./T1
=== Skill Week 5 - Task 1 ===
Current Directory and Process Exit Handling

Current Working Directory:
/home/madhulika/OSSP/skill/week5/task1

Directory Path Displayed Successfully.
Current state saved successfully.

Program is running...
Press Ctrl+C to request process exit.
^C
SIGINT received. Exit request detected.
Processing exit request...

Cleanup completed.
State saved to state.txt
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ ls
T1 directory_state.c state.txt task1.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ cat state.txt
Program state saved successfully.
Process cleanup completed.
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ ./T1 &
[1] 5474
=== Skill Week 5 - Task 1 ===
Current Directory and Process Exit Handling

Current Working Directory:
/home/madhulika/OSSP/skill/week5/task1

Directory Path Displayed Successfully.
```

```
Program is running...
Press Ctrl+C to request process exit.
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ pgrep T1
5474
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ kill -TERM 2456
-bash: kill: (2456) - No such process
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ kill -TERM 5474

SIGTERM received. Exit request detected.
Processing exit request...

Cleanup completed.
State saved to state.txt
[1]+ Done ./T1
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ cat state.txt
Program state saved successfully.
Process cleanup completed.
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$ ls -l
total 32
-rwxr-xr-x 1 madhulika madhulika 16584 Sep 16 17:36 T1
-rw-r--r-- 1 madhulika madhulika 2308 Sep 16 17:35 directory_state.c
-rw-r--r-- 1 madhulika madhulika 61 Sep 16 17:40 state.txt
-rw-r--r-- 1 madhulika madhulika 2054 Sep 16 17:32 task1.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task1$
```

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <limits.h>
#include <signal.h>
#include <string.h>

#define STATE_FILE "state.txt"

volatile sig_atomic_t exit_requested = 0;

void cleanup_resources(void)
{
    FILE *file;

    file = fopen(STATE_FILE, "w");

    if (file != NULL) {
        fprintf(file, "Program state saved successfully.\n");
        fprintf(file, "Process cleanup completed.\n");
        fclose(file);
    }

    printf("\nCleanup completed.\n");
    printf("State saved to %s\n", STATE_FILE);
}

void handle_exit_request(int signal_number)
{
    if (signal_number == SIGINT) {
        printf("\nSIGINT received. Exit request detected.\n");
    } else if (signal_number == SIGTERM) {
        printf("\nSIGTERM received. Exit request detected.\n");
    }

    exit_requested = 1;
}

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```

    if (file == NULL) {
        perror("Unable to save state");
        return 1;
    }

    fprintf(file, "Current Working Directory: %s\n", current_directory);
    fprintf(file, "Program State: Running\n");

    fclose(file);

    printf("Current state saved successfully.\n");

    printf("\nProgram is running...\n");
    printf("Press Ctrl+C to request process exit.\n");

    /* Wait for an exit request */
    while (!exit_requested) {
        sleep(1);
    }

    /* Update state before exiting */
    file = fopen(STATE_FILE, "a");

    if (file != NULL) {
        fprintf(file, "Program State: Exit requested\n");
        fclose(file);
    }

    /* Cleanup resources */
    atexit(cleanup_resources);

    printf("Processing exit request...\n");

    return 0;
}

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

Observation: The program successfully retrieved and displayed the current working directory and saved the program state into a file. During execution, the program continuously waited for an exit request. When Ctrl+C was pressed, the program detected the SIGINT signal, and when SIGTERM was sent using the kill command, the program detected the termination request. After receiving the exit request, the program performed the required cleanup operation, saved the state information, and terminated successfully. The process was also verified using the pgrep command to confirm that it had exited correctly.

Task2: To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

```
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5$ mkdir -p task2
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5$ cd task2
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task2$ nano export_env.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task2$ gcc export_env.c -o T2
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task2$ ls -l
total 28
-rwxr-xr-x 1 madhulika madhulika 16816 Sep 16 17:46 T2
-rw-r--r-- 1 madhulika madhulika 3038 Sep 16 17:45 export_env.c
-rw-r--r-- 1 madhulika madhulika 2509 Sep 16 17:19 task2.c
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task2$ ./T2
=== Skill Week 5 - Task 2 ===
Environment Variable Export

export> export NAME=Madhulika
Environment variable updated successfully.
Name : NAME
Value : Madhulika
export> export COLLEGE=KLH
Environment variable updated successfully.
Name : COLLEGE
Value : KLH
export> export NAME=Student
Environment variable updated successfully.
Name : NAME
Value : Student
export> test
Enter variable name to test: NAME
Child Process: NAME=Student
Child process completed.
export> exit

Program terminated successfully.
madhulika@LAPTOP-D2M8NKBH:~/OSSP/skill/week5/task2$
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

int is_valid_name(const char *name)
{
    int i;

    if (name == NULL || name[0] == '\\0')
        return 0;

    if (!(isalpha((unsigned char)name[0]) || name[0] == '_'))
        return 0;

    for (i = 1; name[i] != '\\0'; i++) {
        if (!(isalnum((unsigned char)name[i]) || name[i] == '_'))
            return 0;
    }

    return 1;
}

void process_export(char *command)
{
    char *argument;
    char *equal_sign;
    char *name;
    char *value;

    argument = command + 6;

    while (*argument == ' ')
        argument++;

```

^{^G} Help ^{^O} Write Out ^{^F} Where Is ^{^K} Cut ^{^T} Execute
^{^X} Exit ^{^R} Read File ^{^\\} Replace ^{^U} Paste ^{^J} Justify

```

printf("=== Skill Week 5 - Task 2 ===\\n");
printf("Environment Variable Export\\n\\n");

while (1) {
    printf("export> ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
        break;

    command[strcspn(command, "\\n")] = '\\0';

    if (strcmp(command, "exit") == 0)
        break;

    if (strncmp(command, "export ", 7) == 0) {
        process_export(command);
    } else if (strcmp(command, "test") == 0) {
        printf("Enter variable name to test: ");
        fflush(stdout);

        if (fgets(variable_name, sizeof(variable_name), stdin) != NULL) {
            variable_name[strcspn(variable_name, "\\n")] = '\\0';

            if (is_valid_name(variable_name)) {
                test_child_process(variable_name);
            } else {
                printf("Error: Invalid variable name.\\n");
            }
        }
    } else {
        printf("Invalid command.\\n");
        printf("Use: export NAME=value, test, or exit\\n");
    }
}

printf("\\nProgram terminated successfully.\\n");

```

^{^G} Help ^{^O} Write Out ^{^F} Where Is ^{^K} Cut ^{^T} Execute
^{^X} Exit ^{^R} Read File ^{^\\} Replace ^{^U} Paste ^{^J} Justify

Observation: The program successfully parsed the export NAME=value syntax and updated environment variables using the setenv() function. Valid variable names were accepted, while invalid names and incorrect export syntax were rejected with appropriate error messages. Existing environment variables were successfully updated with new values. The program also created a child process using fork() and verified that the exported environment variable was available to the child process. The different test cases confirmed that the export functionality and validation were working correctly.